

12

Laboratory 12

12	Thermovision	113
12.1	Simple thermal image analysis	
12.2	Using a neural network based detector for RGB image analysis + thermal imaging (YOLOv3)	
12.3	Object detection using a probabilistic pattern	

12. Thermovision

Thermal images have the advantage that objects, such as people or animals, are usually quite visible (i.e. stand out from the background). The exception is when the background temperature is close to that of the object – for example, on a very warm day. Another problematic situation is clothing that insulates well from outside conditions – for example, a down jacket.

The exercise will explore three approaches to the issue:

- simple object detection using thresholding and object analysis,
- the use of a deep convolutional neural network and fusion of RGB and thermal imaging data,
- using a probabilistic model (non-obligatory exercise).

12.1 Simple thermal image analysis

Exercise 12.1 Instructions for the task:

In this task you will implement a simple image-processing pipeline to extract the objects of interest and determine their bounding boxes. Follow the steps below.

1. Apply binarisation, filtering, and object indexing (labelling).
2. Keep only objects with the required height-to-width proportions.
3. Merge detected parts into a single bounding rectangle (bounding box).
4. An example output is shown in Fig. 12.1.

R Please assume that the implementation of this stage of the exercise should not take more than 30-35 minutes. In other words, please do not spend too much time developing an algorithm for combining silhouettes (even at the cost of its effectiveness).

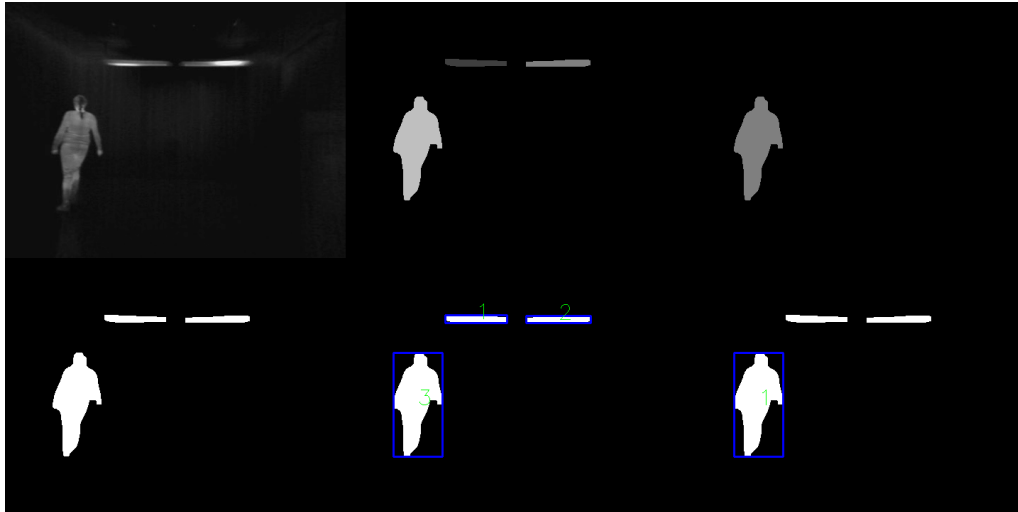


Figure 12.1: An example of the result of the algorithm.

12.2 Using a neural network based detector for RGB image analysis + thermal imaging (YOLOv3)

The first of the approaches to thermal image analysis made it possible to become familiar with the appearance and characteristics of this type of images. Thermovision is a valuable source of information that can serve as an additional data channel when analysing RGB images. Fusing data from RGB and thermal images makes it possible to improve the effectiveness and accuracy of detection, which can be exploited by using a detector based on a deep convolutional neural network.

When using neural networks, data fusion can be carried out in several ways:

1. early fusion — combining data at the beginning of the detection process. The thermal and RGB images are merged into a single image, which is then fed to the detector that determines the bounding boxes around objects and their classes.
2. middle fusion — the detector initially processes the images separately, but at a certain stage of the neural network structure, the feature maps representing both images are combined. Finally, at the network output, we obtain bounding boxes together with their corresponding classes.
3. late fusion — in this case two detectors each receive a single image as input. At their outputs, separate results are obtained, which then need to be combined, e.g. by finding corresponding bounding boxes and averaging their parameters, such as position or size.

This exercise will use and compare approaches based on early and late fusion. In order to perform this task, it is necessary to download a set of images from the test base, the program template, as well as all files containing the weights and configurations of the trained YOLOv3 detectors, which are available for download on the UPEL platform.

Exercise 12.2 How to run the neural network.

1. In the `thermo_detection.py` file, set the paths to `test_rgb` and `test_thermal` (*TODO*).
2. In the `METHOD` section, choose the mode: `FUSION = "EARLY"` or `FUSION = "LATE"`.
3. **Early fusion:** OpenCV loads channels as BGR, so leave B and G unchanged, and set the R channel to the maximum value of the R channel and the greyscale

thermal image (*TODO1*).

4. **Late fusion:** run two detectors (RGB and greyscale thermal), match the bounding boxes and average their parameters to obtain a single bounding box for each pedestrian.
5. Perform bounding box matching using IoU (*Intersection over Union*, Fig. 12.2); implementation in *TODO2*.
6. Algorithm outline: compute IoU for all pairs of boxes **Rect1**–**Rect2** (for $\text{IoU} > 0$), sort them in descending order and greedily select pairs so that each box appears at most once; for the selected pairs, average the coordinates/sizes (cast to **int**) and save to **boxes**.
7. Run the program and compare the performance of both methods.

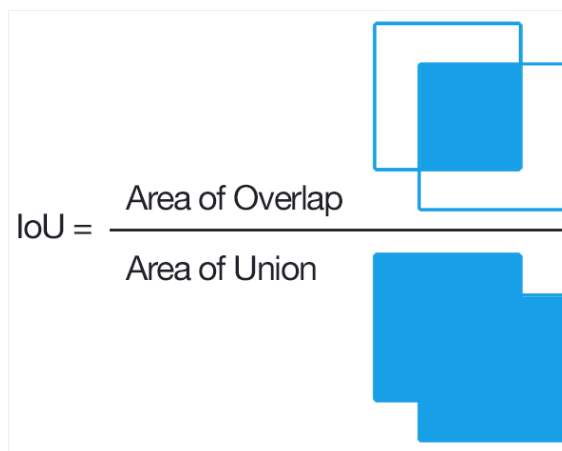


Figure 12.2: *Intersection over Union*. Source: <https://en.wikipedia.org>

12.3 Object detection using a probabilistic pattern

Another approach to pedestrian detection is to observe that the silhouettes of standing people have a fairly distinctive and repeatable appearance. This makes it possible to create a silhouette pattern and then search for it in the image using the sliding window technique.

Exercise 12.3 Non-obligatory assignment. Probabilistic pattern.

A. Saving human silhouettes to a *.png* file.

In the first step, you will need a database of silhouette snippets with specific sizes – 192×64 px. To obtain it, we will use the application from the first part of the exercise: extend it to crop a Region of Interest (ROI) and save the verified silhouettes to *.png* files.

1. We build the dataset by cropping the Region of Interest (ROI) from the image.
2. We save each extracted ROI to a separate *.png* file.
3. We manually select about 30–50 images. In general, they should be frontal, but side views are also acceptable.

B. Creating a silhouette pattern.

1. In the next step, in a separate script, the selected images must be loaded, scaled, and used to determine the template. We scale them to a size of 192×64 pixels. The probabilistic template is created according to the rule $PDM = PDM + B$ (the sum of binary images), where B must be binary. We normalise the obtained template by dividing it by the number of images.
2. We save the resulting template to a file. The template should look similar to the one in Fig. 12.3.

C. Object detection.

1. Load the sequence (as in stage 1). First, test the method on a single frame (e.g. Fig. 12.4).
2. Load the template (optionally convert it to greyscale), convert it to *float32* and divide by the number of images (scale 0–1). Denote it as PDM_1 and compute $PDM_0 = 1 - PDM_1$.
3. Binarise the test frame and apply a sliding window of size 192×64 (the step may be > 1). For the binary mask (B) compute:

$$\sum_y \sum_x (B(y, x) * PDM_1(y, x) + (1 - B(y, x)) * PDM_0(y, x)) \quad (12.1)$$

4. For correct visualisation, normalise the results:

```
result = result / np.max(np.max(result))
result_uint8 = np.uint8(result*255)
```

5. Find local maxima in `result`. The simplest way: choose the maximum while scanning the windows (this will give only one hit) and draw a bounding box.
6. For multiple detections: set a lower threshold and apply elimination of already visited regions (*Non-Maximum Suppression*).
7. To consider: currently you detect only one scale (192×64). How can you add multi-scale detection? Write a comment in the code or implement it.



Figure 12.3: Example pattern.



Figure 12.4: Example frame.